P2 实施指南：KV-Cache 感知推理网关 KVRouter

P2 实施指南：KV-Cache 感知推理网关（代号 KVRouter）— v3 重构版

> 2026-10-05 v3 重构（导师与学员讨论定稿）：v1/v2 里那种"功能列表式的自研网关"定位已经作废——到 2026 年，外部生态已经把其中大部分功能产品化了。本文改用"**独立复现 + 量化测量 + 把前沿改进回馈上游**"的思路重新组织。项目的结构总览见 `03-项目结构定稿.md` （v3），本文是 P2 的详细实施指南。 配套文档： `07-开源贡献指南.md` （本项目与该指南的阶段 3「自产需求 PR」合流）。这份指南的目标和以前一样，希望你看完之后"照着就能开工"。

1. 项目介绍

电梯陈述：你要独立复现 vLLM 前缀路由的两种信息源机制（消费引擎 KV 事件的"精准模式"与依靠网关路由历史的"估算模式"），在同一个负载台上，把它们和真实部署起来的 AIBrix 放在一起做三方对照测量；在此基础上，定位并攻研一个上游目前还没有解决好的前沿问题——**弹性伸缩期的缓存冷启动协同**，等改进得到数据验证之后，再回馈给 AIBrix。

三重定位，三者缺一不可：

1. **复现者**：两种路由信息源你都要亲手实现一遍。厂商只会发布结论（"精准模式更准"），不会发布"到底有多准、在什么负载形态下两者差距会消失"的曲线——这条曲线由你来发布。
2. **测量者**：你自己的实现必须和**真实部署的 AIBrix** 在同一套负载下对照。这相当于 P2 版的"JuiceFS 上秤"：离开了对照组，一切性能声明都只是自说自话。
3. **前沿贡献者**：最终的交付物不能停留在"我仓库里有个功能"，你要做到的是"**我给 AIBrix 提的 issue 里带着测量数据，我提的 PR 里有被接受的修复**"。

这个项目明确不做什么：我们不做"一个比 AIBrix 更好的路由器"（这条功能赛道上我们必输）；不重造 Envoy 数据面；也不改引擎内核。

2. 问题与动机（含 2026-10 生态现勘）

2.1 外部事实（2026-10-05 核实，这些事实构成立项的前提）

- **vLLM v0.10 及以上版本已经支持 KV Cache 事件上报**：引擎会主动把缓存块的分布以事件消息推送给网关。旧版设计中"vLLM 不会告诉你它缓存了什么，信息源只能来自路由历史"这个前提**已经作废**。
- **两种模式在行业里已经有了正式命名**：阿里云 ACK 文档明确区分了**精准模式**（消费引擎块事件）与**估算模式**（网关靠路由历史推断，与引擎解耦）。旧版的 KVRegistry 就属于较弱的估算模式。
- **AIBrix（vllm-project）已经把这件事产品化**：v0.4（2025-08）提供了 KVCache Connector 与 KV Event Subscription；v0.7（2026-06）又加入了可加权的混合软评分路由、高可用网关（Redis 状态同

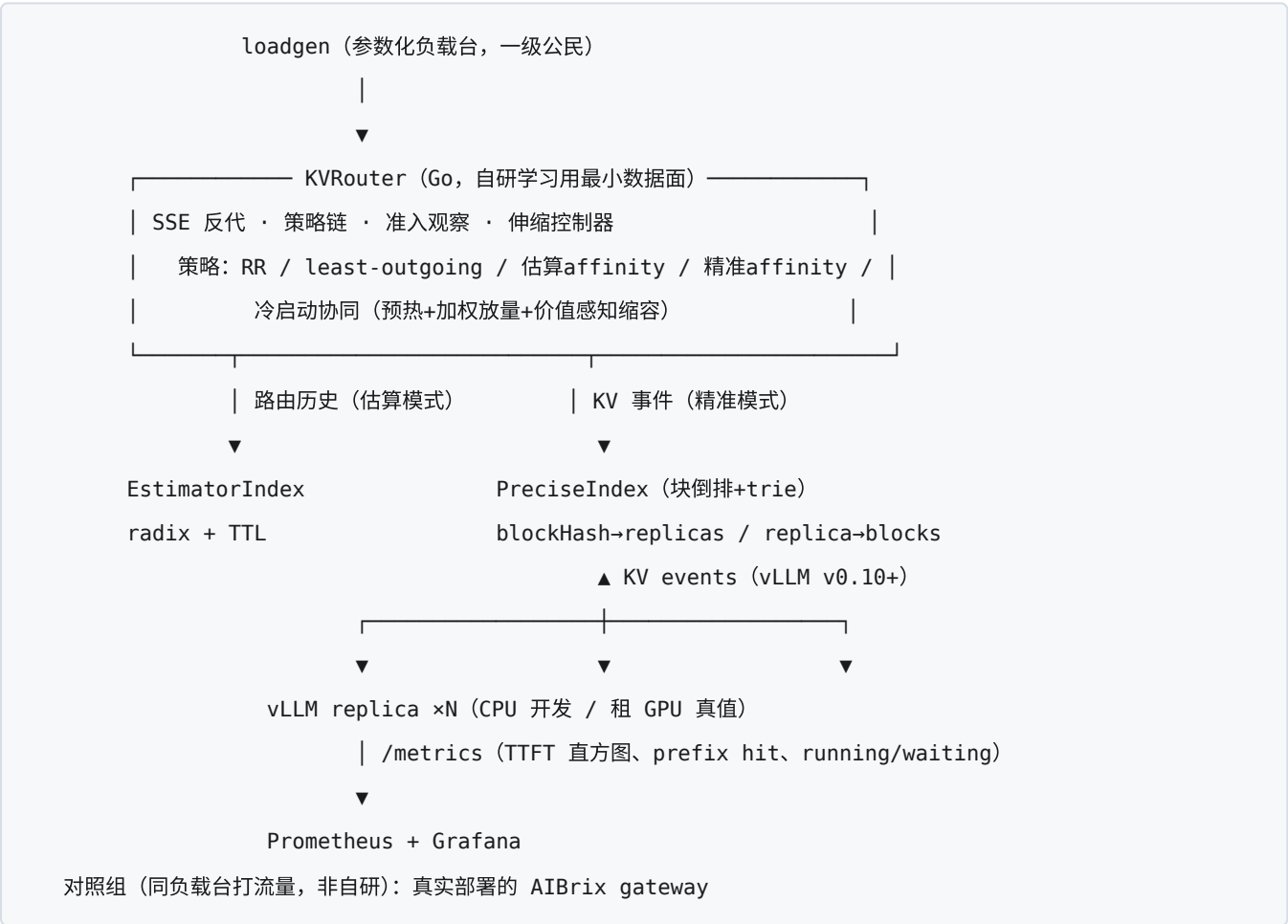
步）、请求优先级与限流、KV offload、P/D 分离（SHFS/NIXL/Mooncake 连接器），三个月内合入了 242 个 PR。它的前缀索引采用 block-based 加 recursive hash（大约 256 token 一个 block）再加 trie。

- **llm-d**（由 Red Hat、Google、IBM、NVIDIA 联合发起，2026-03 进入 CNCF sandbox）同样实现了事件路由、全局块索引和分层 offload；**vLLM Production Stack** 也早就提供了 prefix-aware 和 KV-aware 路由教程。
- 这些事实意味着：前缀路由本身已经成了一件大众化的 commodity 能力；"自研 radix 加 TTL"不再构成差异化，它只是每个合格候选人都应该理解的基线机制。

2.2 仍然成立的问题（这就是本项目要钻的缝）

1. **伸缩期的缓存协同还没有人做好**：新副本的 KV 全冷，前缀感知路由天然不会把流量导给它，于是它热身极慢，甚至可能被饿死；缩容时应该先杀谁、会损失多少前缀缓存、应该怎样 drain，公开资料里基本是空白。软评分中 resident KV 的权重还可能进一步加剧冷副本饿死。
2. **精准模式与估算模式的差距没有公开曲线**：这条差距随"共享前缀比例 × 缓存欠配程度"如何变化？又在什么负载下，根本不值得为精准模式承担与引擎耦合的代价？
3. **流式请求的故障语义很薄弱**：streaming 请求不具备幂等性，副本在输出中途崩溃之后能不能安全重试，现有文档和实现普遍语焉不详。

3. 架构总览



4. 阶段 0：现勘（与使用阶段合并，2–3 周，闸门制）

这是整个项目的地基，也是开源贡献指南阶段 0 在 P2 侧的交付物。

动作清单：

1. 用 vLLM 的 CPU 后端跑通多副本（Qwen2.5-0.5B-Instruct），通读启动参数中与 KV 池、prefix caching、**KV 事件上报**有关的选项；
2. AIBrix 这边**用 quickstart（helm）部署成功就可以，不要求在本地编译整个全家桶**；另外把 Production Stack 的 helm 和 prefix-aware tutorial 跑通；
3. 精读 AIBrix router 源码中的几个部分：软评分、prefix index、KV event 消费、伸缩相关的包；再精读**10 个**最近 merged 的 PR（强制做结构化笔记：diff 长什么形状 / 解决了什么问题 / reviewer 关注什么 / 测试是怎么写的）；
4. 产出 **《P2 对手现勘报告》**：部署命令与版本锁定（镜像 digest）、各系统的 feature 矩阵、复现到的官方 benchmark 数据、**带着证据的缺口排序表**。

头号技术风险（现勘第一周必须关闭，不许带着假设开工）：

> vLLM 的 **CPU 后端，会不会上报与 GPU 后端相同 schema 的 KV 事件**？
>
> - 如果支持，精准模式就可以全程在本地开发；
> - 如果不支持，CPU 阶段的精准模式只能用"符合锁定 schema 的 mock 事件流"来开发逻辑，所有性能真值都要靠租 GPU 获得，预算和排期都要据此重新计算。

缺口闸门：核心 2 最终选哪个缺口，由现勘拿到的证据决定，不能靠今天在这里许愿。

候选缺口	判定它成立需要看到的证据	若被上游提前解决
A（默认押注）伸缩期缓存冷启动：冷副本饿死、热身慢、缩容损失没有度量	在 AIBrix 上人为制造一次扩容事件，新副本在 5 分钟内拿到的流量份额和命中率仍然显著偏低；官方没有预热或放量机制	转做 B 或 C
B 精准与估算的差距曲线	官方没有公开的二维对比（共享比例 × 欠配度）	仍然可以做（复现价值独立存在），但不升为核心 2
C 流式故障安全重试	在 mid-stream 时 kill 副本，出现错误重放或挂死，issue 区无人认领	当 A 被解决时，它升为核心 2

5. 基座设计

5.1 引擎摸底与指标层

- CPU 模式下，用 kind 起 3 个副本，再在本机起 2 个副本；读懂 `VLLM_CPU_KVCACHE_SPACE` 等 KV 池参数并做好笔记；
- **MetricsView 抽象要先行**：vLLM /metrics 的系列名会随版本漂移（TTFT/TPOT 直方图、prefix cache hit/queries、num_requests_running/waiting、cache usage），业务代码只能依赖抽象接口；针对锁定

版本写一份字段映射，并处理好缺字段时的降级；让 metric 名散落在业务代码里，就等于监控系统第一课直接挂掉。

- 所有指标都带上 `mode`（`rr/estimator/precise/aibrix`）、`replica`、`experiment_id` 三个标签；实验配置（模型、vLLM 版本、镜像 digest、负载参数）随报告一起落盘。

5.2 Go 反代基座（自研数据面的全部范围，难点在流式）

SSE 流式转发的五个坑（每一条都要实测之后写进博客）：

1. `httputil.ReverseProxy` 默认会缓冲——你必须把 `FlushInterval` 设为 `-1`，它才会逐 token 下发；
2. 客户端断开时，你必须 cancel 上游的 context，否则上游会继续烧着算力，生成一堆没有人消费的 token；
3. SSE 禁止压缩：如果 `Accept-Encoding` 处理不当，gzip 缓冲会把流式输出攒成"一次性爆发"；
4. idle timeout 是随着每个事件重置的，它按请求总时长来算；
5. **TTFT 的定义是从请求发出到收到第一个内容型 SSE 事件（`data:` 开头且不是心跳帧）的时间**，它既不是首字节时间，也不是收到 header 的时间。

其余的工程点：读写超时分离（`ReadHeaderTimeout` 有值，响应流不设总超时）；每个副本一个连接池，加 `/health` 探针，连续失败就摘除；`X-Request-Id` 全链路透传（核心 2 和增量池都会复用它）。

5.3 负载台（一级公民，所有旗舰图的唯一出口）

它是一个参数化的场景生成器，参数固定写在实验配置文件里，保证三种自研策略与 AIBrix 对照组**吃到完全相同的流量**：

维度	参数
共享前缀	共享比例 f ∈ {0, 0.25, 0.5, 0.8}；system prompt 池大小 M=100，热度服从 Zipf(1.2)
对话形态	单轮短问 / 多轮（轮数服从几何分布，均值为 4）/ 长文档 QA（2KB 固定前缀加各不相同的问题）
到达过程	泊松到达，λ 扫点；突增实验：3 倍 λ 持续 60s
token 形状	输入/输出长度分布参数化（均值/尾部），一切以锁定 tokenizer 的实际切分为准
缓存压力	通过调整 KV 池大小，制造充裕、临界、欠配三档
副本 churn	用脚本按时间表执行 scale +1/-1，供事件研究使用

负载台的输出包括：TTFT/TBT 直方图、吞吐、各副本的 inflight 与 KV usage、前缀命中率（由引擎自报）、新副本拿到的流量份额。**这里有一个陷阱**：一旦客户端自己饱和了，这批数据就作废——所以你要常驻监控 generator 的 CPU 与连接数，最终结论只在"服务端饱和、客户端不满载"的区间里成立。

6. 核心 1：双模式 KV 索引（替换旧增量 1）

6.1 估算模式（亲手复现一条已经被淘汰的路线，这不是凑数）

- 前缀键的取法：把 `messages` 按消息边界取哈希序列 `[h(system), h(user1), h(asst1), ...]`，radix 树的键就是哈希序列的前缀；
- 数据结构：

```
type EstNode struct {
    children map[uint64]*EstNode
    replicas map[ReplicaID]int64 // replica → 最近见到的 unix 秒
    mu       sync.RWMutex
}
```

- 唯一的信息源是网关自己的路由历史；TTL 默认 10 分钟，每次命中就刷新；Pod UID 发生变化或健康状态摆动时，全量摘除对应条目；Registry 自身用 LRU 限制内存；
- **诚实的教学点**：它的误差只会朝"不同文本却有相同 token 前缀"这一个方向漏判（属于保守误差）；但它无法感知引擎内部的淘汰，TTL 本质上是在猜。你只有亲手做一遍，才能把"它为什么会被精准模式取代"讲透。

6.2 精准模式（消费引擎事件，项目的技术核心）

Tokenize 问题（这是最关键的取舍点）：要想把请求内容映射到引擎的块哈希，网关必须先拿到 token ids——这是旧版刻意回避、而精准模式绕不开的成本。你要按顺序实验下面三个方案：

1. **调用引擎的 `/tokenize` 端点**：零新依赖，代价是多一跳 RTT；对 token 序列做 LRU 缓存（同一个 prompt 池的命中率会很高）；先实测这一跳在路由决策耗时中占多大比例；
2. 部署一个 Python tokenizer sidecar（HuggingFace tokenizers）：省下一跳，但引入了一个新组件，还带来模型词汇表如何同步的问题；
3. 在 Go 里内嵌 tokenizer：生态太弱，不选。

默认选方案 1，等数据证明这一跳不可接受时，再升级到方案 2——这个决策过程本身就是上好的面试素材。

块标识（对齐 AIBrix 的公开口径，细节在现勘阶段锁定）：递归哈希 `h_i = H(h_{i-1} || token_i)`，按大约 256 token/block 聚合成块哈希；前缀命中深度等于"请求块哈希链与索引匹配的最长块数 × blockSize"。**事件 schema（字段名、消息类型、推送通道形态）一律以锁定版本的 vLLM 源码或官方文档为准，本文不虚构任何字段名**；现勘报告里必须包含一份 schema 摘录和对应的版本号。

索引结构：

```
type PreciseIndex struct {
    block2rp  map[BlockHash]map[ReplicaID]struct{} // 倒排：块 → 持有副本
    rp2blocks map[ReplicaID]map[BlockHash]struct{} // 正排：副本摘除时 O(k) 清理
    freq      *CountSketch                         // 块热度，核心2预热选 top-K
    mu        sync.RWMutex
}
```

- 事件处理分三类：add、evict、全量重建；从收到事件到索引可见之间的延迟要单独打点（**索引滞后**正是精准模式的主要误差来源）；
- 失配自愈：当你把请求路由到"按理应该有前缀"的副本，引擎却自报 miss（原因可能是事件丢失、滞后或淘汰竞争），这属于性能退化，不算错误；你要按副本监控 miss 率，一旦超过阈值，就触发该副本索引的全量重同步；
- 路由打分仍然可以写成 `score = α·affinity + (1−α)·balance`（affinity 是命中块数的归一化值，balance = 1 − inflight/cap）；**α 扫描降级为教学实验**，它不是旗舰卖点。

6.3 旗舰图 1：精准 vs 估算 vs AIBrix 三方对照

在同一个负载台下，按三种条件（自研估算 / 自研精准 / 真实 AIBrix）× f × 三档缓存压力跑实验，画出 TTFT P50/P99 与命中率的二维曲线。结论允许指向任何方向——包括"f<0.2 时三者没有差异，与引擎耦合并不值得"。**阴性边界比阳性数字更值钱**，因为它回答了"这一招在什么时候没有用"。

7. 核心 2：缓存感知的弹性协同（项目灵魂，默认押注候选 A）

7.1 先测量：伸缩事件研究（event study）

在自研精准路由和 AIBrix 上各做一轮：以 t0 = 新副本 Ready 为原点，取时间轴 −60s 到 +300s，画出：全局 TTFT P50/P99、全局命中率、**新副本的流量份额**、新副本的命中率。你要从中量化两个量：

- **冷启动惩罚**：新副本达到稳态流量份额所需要的时间（饿死的判定标准：5 分钟内份额低于公平份额的 50%）；
- **扩容收益迟滞**：从 scale 决策发生，到全局 P99 实际改善之间的滞后。

缩容侧：在当前常见策略（按 inflight 或副本年龄来杀）下，记录每次被杀掉的副本中独有块所占的比例，也就是**缓存价值损失**。

7.2 设计三件套

(1) 扩容前的热前缀预热

- 从 `PreciseIndex.freq` 中取出 top-K 热块，反推出覆盖这些块所需的最短前缀集合（贪心集合覆盖，K 与预算都参数化）；
- 向新副本发送**合成的 prefill 请求**（长前缀加 `max_tokens=1`），只购买 prefill、不购买 decode；给预热设置 token 预算上限（比如 KV 池的 15%）和速率限制，防止预热抢走在线请求的算力；
- 预热进度要靠块事件来确认，不能假设请求发出去就生效，未确认的块不计入新副本的 readiness。

(2) 加权放量（带迟滞，不要一刀切地切换流量）

新副本就绪之后，不要立刻让它拿满分：

```
w(r) = clamp((readiness − θlo) / (θhi − θlo), 0, 1)   // readiness = 已确认预热块/目标块
```

w 作为乘子进入软评分；权重在 θlo/θhi 之间线性 ramp，如果观察窗口（默认 30s）内份额波动超过阈值，就冻结 ramp 以防止抖动。老副本的 w 恒为 1。

(3) 缓存价值感知缩容 + drain

- 选择副本的方法：`价值(r) = Σ_{b ∈ rp2blocks[r]} 稀缺度(b)`，其中 `稀缺度(b) = 1 / 持有 b 的副本数`——多家共有的块被杀掉也没有损失，独有块的价值最高；优先杀价值低的副本，价值并列时再比较 inflight；
- drain 的流程：先给副本打标记，停止接收新会话；然后等待在途的流式请求结束（超时默认 120s，超时之后只对可以安全重试的请求强切，见增量池 A）；等待期间，该副本的块进入"濒死"状态，亲和打分降权但不归零；
- 防护线：任何时刻，可服务副本数都不得低于按 Little's Law 算出的需求下限；如果某次缩容会突破这条线，就立即中止。

7.3 何时扩容：Little's Law 预测器（配角，不单独作为卖点）

- 到达率用 EWMA 平滑（α=0.3）；`L = λ × (TTFT_预填 + out_tokens × TPOT)`；需求副本数 `N = ceil(L / N_capacity)`，capacity 用"KV usage 达到 90% 时实测的稳态并发"来标定；
- **它与冷启动故事的结合点**：预测的扩容提前量 = 预热耗时（实测值）+ ramp 耗时，这样可以做到"副本 Ready 的时候，热前缀也就绪了"；保留三个对照组：基于 CPU 利用率的 HPA（开发期用它替代 GPU 利用率这个错误信号，报告里要说明等价性论证）、基于 KV usage 的反应式策略、本策略；
- 自己写一个小的 controller，直接打 Deployment 的 scale subresource（避开 prometheus-adapter 这个重依赖）；扩容立即生效，缩容设 5 分钟冷却加两个观察窗口。

7.4 旗舰图 2：伸缩事件时间轴对比

让三种条件（反应式 HPA / 精准路由但无协同 / 完整协同）跑在同一个 churn 脚本下，对比它们的时间轴：TTFT 尖峰的高度与持续时长、命中率凹陷、扩容收益迟滞。**这是整个项目的签名图。**

7.5 终局：回馈上游

先在你自己的仓库里验证原型，然后向 AIBrix 提 issue（贴上 §7.1 的测量数据与复现脚本）；等 maintainer 认可方向之后，再按照它的架构（Envoy 插件或 Go plugin）提交 routing policy 或预热控制器的 PR。**你自己的仓库与上游 PR 属于同一条故事线的两个阶段，不要把它们做成两个不相干的项目。**

8. 增量池（只允许挑 1–2 个，选多了就是范围膨胀）

增量	内容要点	为什么是它
A 流式故障语义与安全重试	用 `X-Request-Id` 做幂等；区分两类故障点：prefill 完成前挂掉（无副作用，可以安全重试）/ 完成后 mid-stream 挂掉（非幂等，靠客户端 last-event 续传或显式重发）；故障注入方式为在随机时刻 kill 副本	成本低、区分度高、上游文档空白；还能和核心 2 的 drain 直接复用
B 多租户缓存干扰测量	透传租户标识；量化"一个租户的大批量流量驱逐了另一个租户的前缀"；软隔离手段是按租户做块价值保护与预热配额	AIBrix 有优先级队列，但还没有人量化过缓存层面的跨租户干扰
C TTFT 归因观测	把 TTFT 拆成四段：排队 / prefill 重算（miss）/ KV 命中 / 网络	它是所有旗舰图解释力的底座，工程很小但分析价值很高

9. 调研层（只写文档或做集成实验，绝不动手自建）

- **P/D 分离与 KV 传输**：NIXL/SHFS/Mooncake、AIBrix PrisKV（单节点无 RDMA 的 P/D）、llm-d 路线；租 GPU 的时候可以选 2 张卡跑一次官方 demo，写一篇观感。你的口径永远是"我调研过，也理解为什么需要 RDMA/高速互连"；
- **KV offload**：用 **LMCache 做一次集成实验**（观察 CPU/SSD 分层对 TTFT 的影响），不要自建 offload 层；
- **Registry HA**：行业答案已经出来了（AIBrix 用 Redis 做状态同步）。本项目诚实地把范围限定为单实例加周期快照，另写一页决策分析（解释为什么第一天就不该上 etcd/Redis：写路径很热、读多写少、允许过期），但不构建。

10. Benchmark 方法学（可复现纪律）

1. **重复次数**：每组实验至少跑 7 次取中位数，并报告 bootstrap 95% CI；只跑 3 次得到的数字不进报告；
2. **冷态控制**：每轮重启 vLLM 副本以清空 KV；热态实验要明确预热流程；轮次之间进程不得残留；
3. **完全锁定**：模型版本、vLLM 版本、镜像 digest、机器配置、负载配置文件全部随报告入库，做到 `make bench COND=xxx` 一键复现；
4. **三条件同流量**：自研的两种模式与 AIBrix 对照组共用同一份 loadgen 输出（固定随机种子的请求轨迹文件）；
5. **CPU 数据要诚实标注**：CPU 模式只对机制与相对趋势负责，绝对值没有生产意义；两张旗舰图最终都要由 GPU 真值替换，报告保留两个版本并说明差异；
6. **客户端哨兵**：负载台的 CPU/连接数监控曲线作为每张性能图的附件，客户端接近饱和的轮次一律作废重跑。

11. 项目优点（面试时可以讲）

1. 你完整走过了"现勘 → 复现 → 测量 → 发现缺口 → 设计 → 回馈上游"这条闭环，并没有靠堆砌功能凑数；
2. 精准与估算两种信息源你都亲手实现过，还做了包含真实 AIBrix 的三方对照，你对 vLLM KV 事件机制的理解可以细到代码级；
3. 你用事件研究法量化了冷启动惩罚，并且用预热、加权放量、价值感知缩容构成了一套完整的协同设计，每个参数都有数据来源；
4. 你手里有 SSE 数据面的硬货（五个坑全部实测过）、可解释的排队论应用（Little's Law，不搞 ML），以及严格的实验纪律；
5. 你的改进最终进入了上游的 issue/PR——这种经过第三方背书的技术判断，是简历上信用等级最高的东西。

12. 诚实清单（不足之处主动说）

- 3 副本的 kind/单机规模只用于机制验证，不做任何规模外推声明；
- CPU 后端是否支持 KV 事件是头号风险（见 §4），mock 事件期间得到的一切精准模式数字，都标注为逻辑验证；
- 块事件 schema 跟随 vLLM 版本变化，升级成本真实存在（MetricsView 与事件适配层就是为它准备的）；
- 单个 Registry 不做 HA（这是刻意划定的范围，行业答案引用 AIBrix 的 Redis 方案）；
- 我们不声称能控制引擎内部的淘汰；预热是一种"请求级"手段，会受到引擎调度的影响，预热是否完成以事件确认为准，不能靠假设；
- 核心 2 押注候选 A，天然带着"被上游提前解决"的风险，§4 的闸门会保护你及时转向，不要硬撞；
- 没有 RDMA 和多机 P/D 的实测数据：只做到调研级；GPU 真值只覆盖 1–2 次租卡窗口。

13. 砍项清单（v2 → v3，说清楚为什么砍）

v2 内容	处置	理由
SLO 分级/准入/抢占（旧增量 4）	砍进增量池，默认不做	AIBrix v0.7 已经有 priority tier 加 rate limiting，重造只会扣分
把预测式弹性作为独立卖点	降级为核心 2 的一个组件	单独拿出来做，充其量只是"又一个 HPA 替换品"
会话亲和（旧增量 7）	删除	在精准事件索引下它基本是个伪问题；"要在路由层软化"这件事本身就说明它不该存在
构建 Registry HA	只写决策分析，不构建	行业答案已经产品化
把 α 双目标 Pareto 当旗舰图	降为核心 1 的教学实验	weighted soft-scoring 已经产品化，没有辨识度

v2 内容	处置	理由
自建 P/D 分离、自建 KV offload	降为调研/LMCache 集成	需要多卡与高速网络，门槛高，上游投入重，自建必输
SSE 反代	**保留在基座中**	它是唯一具有训练价值的自研数据面

14. 实施步骤（10 步，闸门驱动）

步骤	内容	验收 / 注意点
0 现勘	§4 的完整清单：CPU 多副本、AIBrix/ProdStack quickstart、10 个 PR 笔记、现勘报告、关闭 CPU 事件风险、用缺口闸门定下核心 2	现勘报告评审通过之后才能进入步骤 1；**轻量部分立即并行启动，不等 P1 完工**
1 基座	SSE 反代（五个坑全测）+ MetricsView + 负载台 + Grafana	流式对话稳定，杀掉后端不会拖垮连接；负载台可以切换三种策略
2 基线报告	RR 对比 least-outgoing，标定负载台的工作区	客户端不饱和的证据随报告附上
3 估算索引	§6.1 的 radix + TTL + 摘除	机制正确，文档里标注"对已淘汰路线的复现"
4 精准索引	tokenize 方案选型实测 + 事件消费 + 双倒排 + 失配重同步	索引滞后有打点；事件 schema 锁定版本
5 旗舰图 1	三条件 × f × 缓存压力对照（包含真实 AIBrix）	二维曲线加 95% CI；阴性边界写清楚
6 冷启动测量	§7.1 的事件研究（自研与 AIBrix 各一轮）	饿死、迟滞、价值损失三组量化 ← **issue 素材**
7 协同设计	预热 + 加权放量 + 价值感知缩容 + drain	旗舰图 2：尖峰、凹陷、迟滞的三条件对比
8 增量池	只挑 1 个（默认 A：流式故障语义）	一份独立的小报告
9 真值校准	租 GPU（Qwen 7B 级别）重跑旗舰图 1/2；可选做 LMCache 与 2 卡 P/D demo	用真数据替换 CPU 数据，两个版本都保留
10 回馈	提 issue（带数据）→ 讨论 → 提 PR（遵循 AIBrix 的架构）	至少 1 个代码级 PR；即使被拒，也保留"为什么被拒"的完整故事

拼接逻辑：1+2 得到一个可测量的基座；3+4+5 构成双模式复现（第一个完整故事）；6+7 构成前沿纵深（第二个故事，也是 PR 的来源）；9 用真值收口；10 把信用落到上游。你在任意一层停下来都能写简历，但秋招包的目标是完成 1–7 加 10。

15. 排期与死线（倒推）

- **立即开始（与 P1 并行）**：做步骤 0 的轻量部分（vLLM CPU + AIBrix quickstart + 读码），每周投入不超过 0.5 天，不抢 P1 的主线时间；
- P1 收尾之后全职做 P2：步骤 1–5 大约需要 9–11 周，6–7 大约 6–8 周，8 大约 2–3 周，9 大约 1 周；
- **2027 年 8 月秋招包的最低构成**：基座 + 核心 1（含旗舰图 1）+ 核心 2 主实验（旗舰图 2 初稿）+ 至少 1 个 AIBrix 代码级 PR（docs/test PR 不计入，必须是代码级；如果 PR 流程还没闭合，至少要做到 issue 有 maintainer 的正式回应）；
- 砍项顺序（时间不够时）：增量池 → GPU 校准里的 demo 类部分（旗舰图校准不能砍）→ 核心 2 的 drain 细化；核心 1 与旗舰图 1 不砍。

16. 预期成果

交付物：KVRouter 仓库（代码、README、设计决策记录 ADR、复现命令）、负载台（独立可用，也可以拿来打 AIBrix）、现勘报告、两份旗舰图实验报告、两篇博客（一篇讲 SSE 反代五个坑，一篇讲精准与估算模式的复现笔记）、AIBrix issue 与 PR 链接。

简历条目草稿（V1 主版）：

> 独立复现并量化了 vLLM 前缀路由的两种信息源机制（引擎 KV 事件的精准模式与路由历史的估算模式），在同一负载台下与真实的 AIBrix 做三方对照，给出了收益随共享前缀比例与缓存欠配度变化的边界曲线；通过事件研究定位并量化了弹性伸缩期的 KV 冷启动惩罚（冷副本饿死与扩容迟滞），设计了热前缀预热、加权放量、缓存价值感知缩容的协同策略，使伸缩期 TTFT 尖峰下降 X%、收益迟滞缩短 Y%；相关测量与改进以 N 个 issue/PR 回馈给 AIBrix（至少 1 个代码级 PR 已 merged）。